Session 3: Keypoints, Descriptors, Matching, Homography

Notebook: `Session3.ipynb`.

Order used here: this follows the notebook's order, which is already logical.

1. What a keypoint is
2. Detectors: Harris → SIFT → ORB
3. Descriptors
4. Matching: brute force, FLANN and the ratio test
5. Homography and image alignment
6. A comparison of all the detectors

> 🎯 This session isn't needed for the flag detector. A flag is a plain-colored region with no texture, so it has almost no keypoints. It's useful general knowledge, and it's the basis of your panorama study plan.

---

1. What a keypoint is (cells 1–8)

💡 Picture a small window sliding over the image:

Region	Shifting the window…	Useful?
Flat	changes nothing in any direction	❌ It could be anywhere
Edge	changes things only *across* the edge	⚠️ You can slide along it and it looks the same
Corner	changes things in **every** direction	✅ It's unique and easy to find again

So the best **keypoints** are corners and blob-like spots: places you could find again in another photo of the same scene.

---

2. Detectors

Cell 13: imports and `show()`

This is the same helper as before. It converts BGR → RGB for color images and uses a gray colormap for 2-D ones.

Cells 14–27: Harris corner detector

```
img  = cv.imread(r'images/chessboard.jpg')
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
gray = np.float32(gray)
dst  = cv.cornerHarris(gray, 2, 3, 0.04)
```

- `r'...'` is a **raw string**, so backslashes in Windows paths aren't treated as escape codes.
- `cornerHarris` requires a **float32** gray image, which is why `np.float32(gray)` is needed.
- The parameters:
  - `blockSize=2`: the size of the neighbourhood examined around each pixel.
  - `ksize=3`: the size of the Sobel kernel used for the gradients (Session 2).
  - `k=0.04`: a sensitivity constant, normally 0.04–0.06.
- 💡 For each pixel, Harris summarizes the gradients in its neighbourhood into a 2×2 matrix M and computes the score `R = det(M) − k·trace(M)²`:
  - **R large and positive** means a corner.
  - **R negative** means an edge.
  - **R near 0** means a flat area.
- The output `dst` is a float image of these R values, one per pixel.

```
dst = cv.dilate(dst, None)                # only to make the dots bigger for display
img1 = img.copy()
img1[dst > 0.2*dst.max()] = [0, 0, 255]   # paint strong corners red (BGR)
```

- `dilate` here **isn't part of the detection**. It just enlarges the points so you can see them.
- `dst > 0.2*dst.max()` is a **boolean mask** that is `True` wherever the score is above 20% of the strongest corner. Indexing with it paints those pixels.
- `.copy()` keeps the original image untouched.
- A lower fraction such as 0.01 finds more, weaker corners. A higher one keeps only the strongest.
- ⚠️ The markdown in cell 27 mentions `0.02` and "the other three images", which are leftovers from another tutorial. The code uses `0.2`.
- ⚠️ Harris is **not scale-invariant**. A sharp corner, zoomed in, becomes a gentle curve that Harris no longer sees as a corner.

Cells 28–38: SIFT (Scale-Invariant Feature Transform)

```
img = cv.imread("images/eiffel1.jpg", cv.IMREAD_GRAYSCALE)
sift_Detector = cv.SIFT_create()
sift_keypoints, descriptors = sift_Detector.detectAndCompute(img, None)
img_sift = cv.drawKeypoints(img, sift_keypoints, None)
```

- ⚠️ The markdown says you need an old OpenCV for SIFT. That's **outdated**: the patent expired in 2020, and `cv.SIFT_create()` has been in normal OpenCV since version 4.4.
- 💡 How SIFT works:
  1. **Scale space:** blur the image more and more and shrink it (a Gaussian pyramid), then take the **DoG** between neighbouring blur levels (Session 2, part 5).
  2. **Find extrema:** points that are a max or min compared with their neighbours in position *and* in scale. This is what makes it **scale-invariant**.

3. **Clean up:** remove points with low contrast and points that lie on edges.
4. **Orientation:** take the dominant gradient direction around the point and rotate everything to it. This makes it **rotation-invariant**.
5. **Descriptor:** a 4×4 grid of cells × an 8-bin gradient histogram = **128 numbers**.

- `detectAndCompute(image, mask)` : `mask=None` searches the whole image.
  - It returns `keypoints`, a list of `KeyPoint` objects, each with `.pt` (x,y), `.size`, `.angle` and `.response` (strength).
  - It also returns `descriptors`, an array of shape `(N, 128)` in **float32**.
- `drawKeypoints(img, kps, None)` draws small circles. Add `flags=cv.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS` to see each keypoint's size and orientation.

Cells 39–46: ORB (Oriented FAST and Rotated BRIEF)

```
ORB_object = cv.ORB_create()
keypoints = ORB_object.detect(img)
keypoints, descriptors = ORB_object.compute(img, keypoints)
imageresult = cv.drawKeypoints(img, keypoints, None, color=(255,0,0), flags=0)
```

- This shows the two steps separately (`detect`, then `compute`). `detectAndCompute` does both in one call.
- 💡 How ORB works:
  - **FAST** finds corner candidates very quickly by checking a circle of 16 pixels around each point.
  - It ranks them with the Harris score and keeps the best ones (default 500).
  - It computes an **orientation** for each one (the intensity centroid).
  - **BRIEF** builds the descriptor: 256 yes/no comparisons ("is pixel A brighter than pixel B?") = 256 bits = **32 bytes**.
  - The pattern is rotated to match the orientation, so it's **rotation-invariant**. A pyramid gives it some scale invariance.
- It's **free and very fast**, which made it the default choice for real-time use.
- `descriptors` has shape `(N, 32)` and type `uint8` (binary).
- `color=(255,0,0)` on an image that came in gray becomes BGR, so the circles are **blue**.

---

3. Descriptors (cell 35)

💡 A **keypoint** says *where* something interesting is. A **descriptor** is a list of numbers that describes *what it looks like*, so it can be recognized in another image.

- **Repeatability:** the same point should be detected again in another image of the scene.
- **Matchability (distinctiveness):** different points should have clearly different descriptors.
- **Locality:** the descriptor is based on a small neighbourhood, so it survives partial occlusion.

---

4. Matching

Cells 48–51: preparation

```
query_img = cv.imread('./images/query_image.png')
train_img = cv.imread('./images/training_image.jpg')
orb = cv.ORB_create()
query_keypoints, query_descriptors = orb.detectAndCompute(query_img_gray, None)
train_keypoints, train_descriptors = orb.detectAndCompute(train_img_gray, None)
```

- **Query** is the object you're looking for. **Train** is the image or database you search in.

Cells 53–64: brute-force matcher

```
matcher = cv.BFMatcher(cv.NORM_HAMMING, crossCheck=True)
matches = matcher.match(query_descriptors, train_descriptors)
matches = sorted(matches, key=lambda x: x.distance)
img3 = cv.drawMatches(query_img, query_keypoints, train_img, train_keypoints,
                      matches[:numberofmatches], None,
                      flags=cv.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
```

- **Brute force** compares every query descriptor with **every** train descriptor and keeps the closest one.
- The **distance** measure must match the descriptor type:

Descriptor	Norm
ORB, BRIEF, BRISK (binary)	`cv.NORM_HAMMING` : the number of differing bits
SIFT, SURF (float)	`cv.NORM_L2` : straight-line distance

- `crossCheck=True` keeps a pair only if A's best match is B **and** B's best match is A. That's a cheap way to remove bad matches.
- `match` returns a list of `DMatch` objects:
  - `.distance` : lower is better.
  - `.queryIdx` : the index into the **first** descriptor array passed in.
  - `.trainIdx` : the index into the **second** one.
- `sorted(..., key=lambda x: x.distance)` puts the best matches first. `lambda x: x.distance` means "sort by each match's distance".
- `drawMatches` places the two images side by side and draws a line for each match. `NOT_DRAW_SINGLE_POINTS` hides keypoints that weren't matched.
- ⚠️ The comment says "first 10", but `numberofmatches = 20`.

Cells 65–67: FLANN with Lowe's ratio test

```
sift = cv.SIFT_create()
kp1, des1 = sift.detectAndCompute(query_img_gray, None)
kp2, des2 = sift.detectAndCompute(train_img_gray, None)
FLANN_INDEX_KDTREE = 1
index_params  = dict(algorithm=FLANN_INDEX_KDTREE, trees=5)
search_params = dict(checks=50)
flann = cv.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(des1, des2, k=2)
```

- **FLANN** finds *approximate* nearest neighbours using smart search trees. It's much faster than brute force when there are many descriptors.
- A **KD-tree** index suits float descriptors such as SIFT. `trees=5` builds 5 trees in parallel. For ORB you'd use the LSH index instead (`algorithm=6`).
- `checks=50` sets how many tree leaves to examine. More checks are more accurate and slower.
- `knnMatch(k=2)` returns the **two best** candidates for each query descriptor.

```
matchesMask = [[0,0] for i in range(len(matches))]
for i, (m, n) in enumerate(matches):
    if m.distance < 0.7 * n.distance:
        matchesMask[i] = [1, 0]
```

- 💡 **Lowe's ratio test:** a match is trustworthy only if the best candidate (`m`) is **clearly better** than the second best (`n`). If they're close, the point is ambiguous (like a repeated window on a building), so it's rejected.
- `matchesMask` marks which pairs to draw: `[1,0]` draws the best match and hides the second.
- `drawMatchesKnn` draws matches in this k-nearest format, using `draw_params` :
  - good matches in green;
  - single points in blue, which in OpenCV's BGR order is `(255,0,0)` .

---

5. Homography and image alignment

Cells 68–76: theory

💡 A **homography** is a 3×3 matrix H that maps every point in image 1 to its position in image 2:

```
[x2·w, y2·w, w]ᵀ = H · [x1, y1, 1]ᵀ        then divide by w
```

- It has 8 degrees of freedom: 9 numbers, but the overall scale doesn't matter.
- It needs at least **4 point pairs** to compute.
- It's exactly correct only when:
  1. the scene is **flat** (a sheet of paper, a wall, a screen), or
  2. the camera only **rotates about its own center**, which is what you do for panoramas. (The notebook says "optical axis"; the precise condition is rotation about the camera center.)

Cells 77–78: matching the two forms

```
query_image = cv.imread('./images/form.jpg')          # clean template
train_image = cv.imread('./images/scanned-form.jpg')  # tilted scan
MAX_FEATURES = 500
GOOD_MATCH_PERCENT = 0.15
orb = cv.ORB_create(MAX_FEATURES)
keypoints1, descriptors1 = orb.detectAndCompute(train_imggray, None)   # the scan
keypoints2, descriptors2 = orb.detectAndCompute(query_imggray, None)   # the template
matcher = cv.DescriptorMatcher_create(cv.DESCRIPTOR_MATCHER_BRUTEFORCE_HAMMING)
matches = matcher.match(descriptors1, descriptors2, None)
matched = sorted(matches, key=lambda x: x.distance, reverse=False)
numGoodMatches = int(len(matched) * GOOD_MATCH_PERCENT)
matches = matched[:numGoodMatches]
```

- ⚠️ The names are swapped compared with section 4: here, set **1 is the scan** and set **2 is the template**. Keep track of which is which.
- `DescriptorMatcher_create(...)` is another way to create a brute-force Hamming matcher, here without cross-check.
- Only the best 15% of matches are kept.

Cell 79: getting the coordinates

```
points1 = np.zeros((len(matches), 2), dtype=np.float32)
for i, match in enumerate(matches):
    points1[i, :] = keypoints1[match.queryIdx].pt
    points2[i, :] = keypoints2[match.trainIdx].pt
```

- This builds two **N×2 arrays of (x, y)**, where row i of each is the same physical point.
- `queryIdx` indexes `keypoints1`, the first set passed to `match`, and `trainIdx` indexes `keypoints2`.

Cells 80–82: computing and applying H

```
h, mask = cv.findHomography(points1, points2, cv.RANSAC)
height, width, channels = query_image.shape
im1Reg = cv.warpPerspective(train_image, h, (width, height))
cv.imwrite("aligned.jpg", im1Reg)
```

- 💡 **RANSAC** handles bad matches:

  1. Pick 4 random pairs and compute an H from them.
  2. Count how many *other* pairs agree with it (the **inliers**).
  3. Repeat many times and keep the H with the most inliers.

  The result is robust even when many matches are wrong. `mask` marks which pairs were inliers.

- `warpPerspective(src, H, (width, height))` moves every pixel of the scan to its position in the template. The output size is given as **(width, height)**, which is the reverse of `.shape`'s (height, width) order.

- `imwrite` saves the file in the notebook's folder.

- ⚠️ Cell 81 shows `train_image` but titles it "query".

---

6. Comparing detectors (cells 84–85)

Algorithm	Type	Invariance	Descriptor	Speed
Harris	detector	rotation	—	fast
FAST	detector	—	—	very fast
BRIEF	descriptor	—	binary	very fast
ORB	both	rotation, (partly) scale	binary	fast
SIFT	both	rotation, scale	float (128)	slow
BRISK	both	rotation, scale	binary	fast

```
fast = cv.FastFeatureDetector_create()
kp_fast = fast.detect(img1, None)
brisk = cv.BRISK_create()
kp_brisk, des_brisk = brisk.detectAndCompute(img1, None)
```

- The last cell runs all four on the Eiffel Tower image and plots them in a 2×2 grid, converting each from BGR to RGB.
- Harris uses the threshold `0.01*max` here, so it finds many more corners than the chessboard example did.
- FAST finds the **most** points because it doesn't rank or limit them. ORB keeps the best 500.

Session 3 gotchas

1. `cornerHarris` needs a **float32** gray image.
2. `dilate` on the Harris output is only for display.
3. SIFT is in standard OpenCV now; ignore the "old version" note.
4. Use `NORM_HAMMING` for binary descriptors and `NORM_L2` for float ones.
5. `queryIdx` belongs to the **first** argument of `match()`, whatever the variables are called.
6. Sizes passed to `warpPerspective` and `resize` are **(width, height)**.
7. Homography is exact only for planar scenes or pure rotation.